

# Algoritmi avansați

## C11 - Diagrame Voronoi

Mihai-Sorin Stupariu

Sem. al II-lea, 2025 - 2026

Definiție, proprietăți elementare

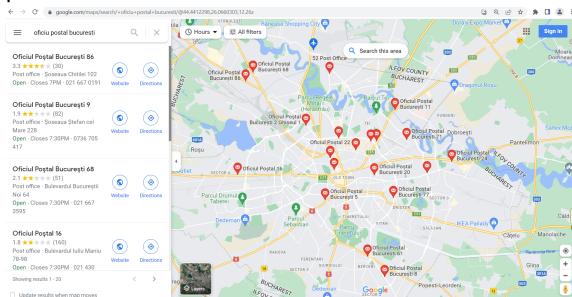
Diagrame Voronoi și triangulări Delaunay

Un algoritm eficient

Subdiviziuni planare. DCEL

# Motivație

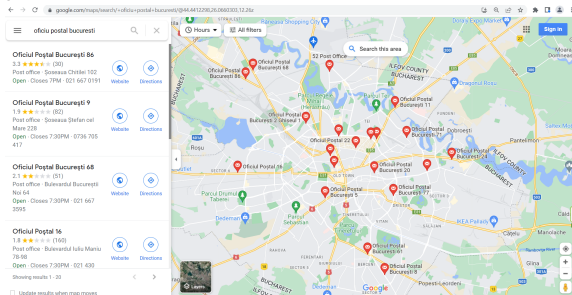
- *Problema oficiilor poștale:* Se consideră o mulțime de puncte (oficiile poștale) din plan. Ce zone vor deservi aceste oficii?



Sursa: Google Maps

# Motivație

- *Problema oficiilor poștale:* Se consideră o mulțime de puncte (oficiile poștale) din plan. Ce zone vor deservi aceste oficii?



Sursa: Google Maps

- Ideea de a delimita “zone de influență” a apărut cu multă vreme în urmă (de exemplu în lucrările lui Descartes, dar și în **legătură cu alte probleme**; este utilizată în mod curent în varii domenii. În plus, astfel de “împărțiri” apar **în natură**. Conceptul aferent este cel de **diagramă Voronoi** - există o multitudine de aplicații.

# Aplicații - rețele de transport

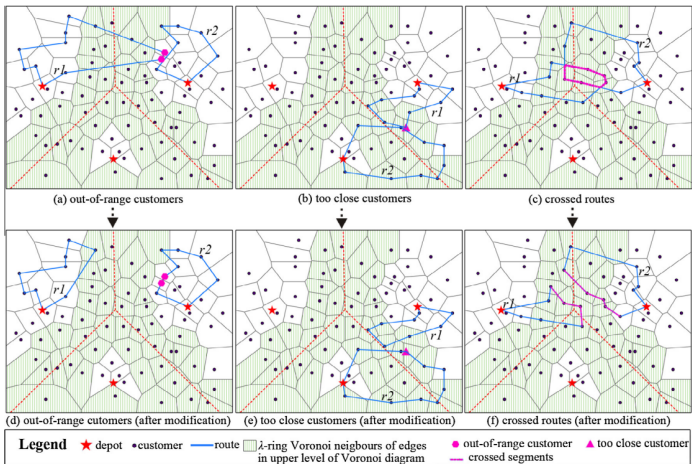


Fig. 3. Inappropriate routes between depots and their modifications.

Sursa: [Tu et al., 2014]

# Aplicații - rețele de transport

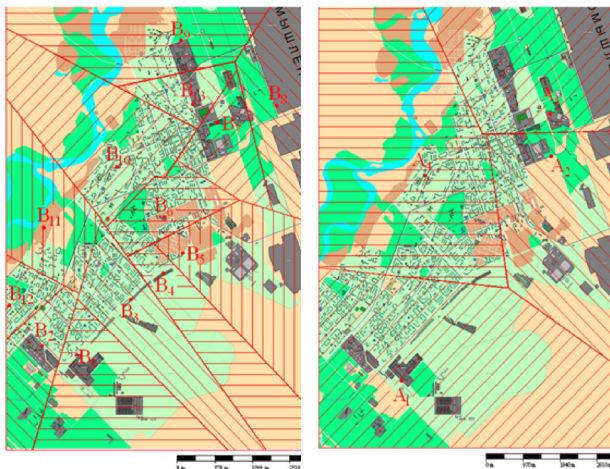


Fig. 4. Voronoi diagram to optimize: (a) the zones serviced by transport terminals of large city districts; (b) freight delivery zones from the logistic centers of a large city.

Sursa: [Lebedeva et al., 2018]

# Aplicații - medicină

ANTIGA *et al.*: COMPUTATIONAL GEOMETRY FOR PATIENT-SPECIFIC RECONSTRUCTION AND MESHING OF BLOOD VESSELS

681

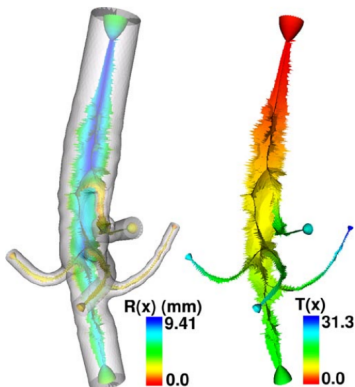


Fig. 8. Voronoi diagram colored according to maximal inscribed sphere radius values  $R(x)$  (left), and solution  $T(x)$  of Eikonal equation from aorta inlet (right) for the abdominal aorta model (Surface: 8096 points, 16188 triangles. Voronoi diagram: 26463 points, 18370 polygons. Voronoi diagram computation time: 10 s. Solution of Eikonal equation time: 8 s).

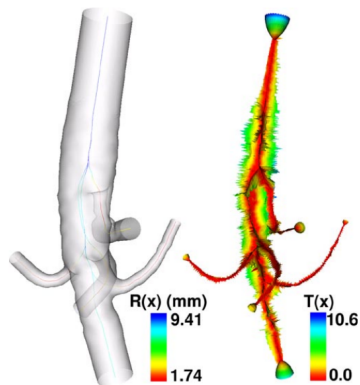


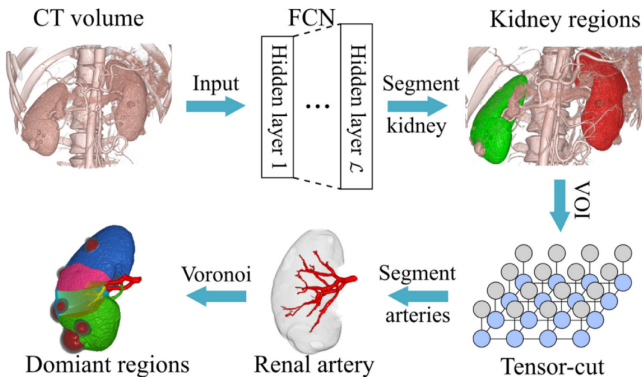
Fig. 9. Calculated central paths (left) and solution of Eikonal equation from central path points (right) for subsequent surface characterization (total five central paths backtracing time: 1 s. Solution of Eikonal equation time: 8 s).  $R(x)$  is maximal inscribed ball radius values defined on central paths, and  $T(x)$  is radial travelttime (since in this case  $F(x) = 1$  everywhere,  $T(x)$  also represents geodesic distance).

Sursa: [Antiga et al., 2003]

# Aplicații - medicină

C. Wang, H.R. Roth, T. Kitasaka et al. / Computerized Medical Imaging and Graphics 77 (2019) 101642

3



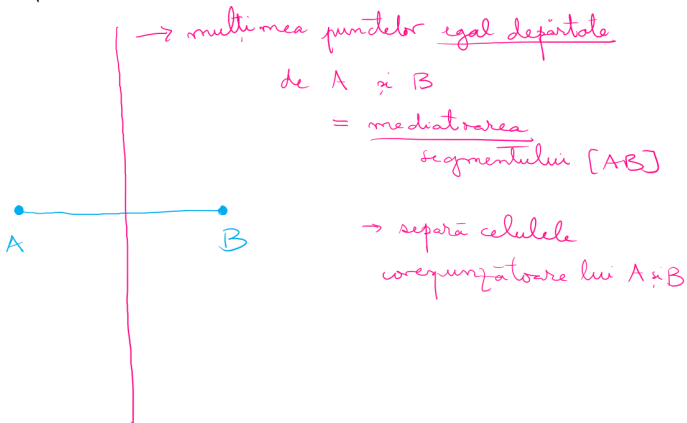
**Fig. 1.** Workflow: Our precise estimation approach can be divided into three parts: kidney segmentation, renal artery segmentation and estimation of vascular dominant regions.

Sursa: [Wang et al., 2019]



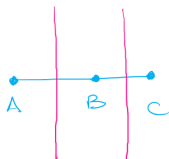
## Exemple - diagrame Voronoi în context 2D

1) 2 puncte distincte

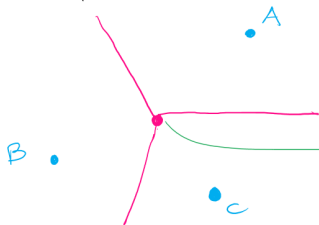


## Exemple - diagrame Voronoi în context 2D

2) 3 puncte distincte, coliniare



3) 3 puncte necoliniare



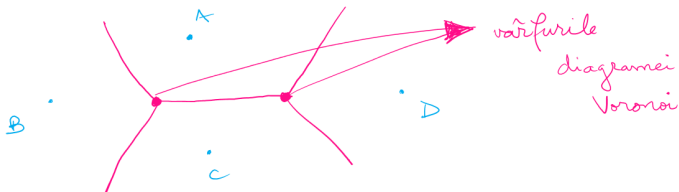
mediatoarele segmentelor

$[AB], [BC], [AC]$

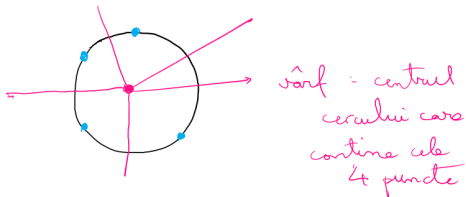
centrul cercului înscris  
 $\triangle ABC$

# Exemple - diagrame Voronoi în context 2D

4) 4 puncte necoliniare, neconcide



4') 4 puncte concide



# Formalizare

- Fie  $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$  o mulțime de puncte din planul  $\mathbb{R}^2$ , numite **situri**.

## Formalizare

- ▶ Fie  $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$  o mulțime de puncte din planul  $\mathbb{R}^2$ , numite **situri**.
- ▶ **Diagrama Voronoi** a lui  $\mathcal{P}$  (notată  $\text{Vor}(\mathcal{P})$ ) este o divizare a planului  $\mathbb{R}^2$  în  $n$  celule  $\mathcal{V}(P_1), \dots, \mathcal{V}(P_n)$  cu proprietatea că

$$P \in \mathcal{V}(P_i) \Leftrightarrow d(P, P_i) \leq d(P, P_j), \quad \forall j = 1, \dots, n.$$

# Formalizare

- ▶ Fie  $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$  o mulțime de puncte din planul  $\mathbb{R}^2$ , numite **situri**.
- ▶ **Diagrama Voronoi** a lui  $\mathcal{P}$  (notată  $\text{Vor}(\mathcal{P})$ ) este o divizare a planului  $\mathbb{R}^2$  în  $n$  celule  $\mathcal{V}(P_1), \dots, \mathcal{V}(P_n)$  cu proprietatea că

$$P \in \mathcal{V}(P_i) \Leftrightarrow d(P, P_i) \leq d(P, P_j), \quad \forall j = 1, \dots, n.$$

- ▶ Două celule adiacente au în comun o *muchie* sau un *vârf* (punct de intersecție a muchiilor).
- ▶ **Atenție!** Vârfurile lui  $\text{Vor}(\mathcal{P})$  sunt diferite de punctele din  $\mathcal{P}$ .
- ▶ Uneori, prin abuz de limbaj, este precizată doar împărțirea în muchii / vârfuri.

# Formalizare

- ▶ Fie  $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$  o mulțime de puncte din planul  $\mathbb{R}^2$ , numite **situri**.
- ▶ **Diagrama Voronoi** a lui  $\mathcal{P}$  (notată  $\text{Vor}(\mathcal{P})$ ) este o divizare a planului  $\mathbb{R}^2$  în  $n$  celule  $\mathcal{V}(P_1), \dots, \mathcal{V}(P_n)$  cu proprietatea că

$$P \in \mathcal{V}(P_i) \Leftrightarrow d(P, P_i) \leq d(P, P_j), \quad \forall j = 1, \dots, n.$$

- ▶ Două celule adiacente au în comun o *muchie* sau un *vârf* (punct de intersecție a muchiilor).
- ▶ **Atenție!** Vârfurile lui  $\text{Vor}(\mathcal{P})$  sunt diferite de punctele din  $\mathcal{P}$ .
- ▶ Uneori, prin abuz de limbaj, este precizată doar împărțirea în muchii / vârfuri.
- ▶ Diagrame Voronoi pot fi construite pentru **diverse funcții distanță** (e.g. **distanța Manhattan**); forma celulelor depinde de **forma “cercului”** în raport cu funcția distanță respectivă. De asemenea, există și **diagrame Voronoi cu ponderi**.

# Structura unei diagrame Voronoi

- Fie  $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$  o mulțime de situri (puncte) din planul  $\mathbb{R}^2$ .



# Structura unei diagrame Voronoi

- ▶ Fie  $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$  o mulțime de situri (puncte) din planul  $\mathbb{R}^2$ .
- ▶ Celula asociată unui punct este o intersecție de semiplane:

$$\mathcal{V}(P_i) = \bigcap_{j \neq i} h(P_i, P_j),$$

unde  $h(P_i, P_j)$  este semiplanul determinat de mediatoarea segmentului  $[P_i P_j]$  care conține punctul  $P_i$ . În particular: fiecare celulă este o mulțime convexă.

Aplicabilitate: algoritm (lent, complexitate-timp  $O(n^2 \log n)$ ) de determinare a diagramei Voronoi.

# Structura unei diagrame Voronoi

- ▶ Fie  $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$  o mulțime de situri (puncte) din planul  $\mathbb{R}^2$ .
- ▶ Celula asociată unui punct este o intersecție de semiplane:

$$\mathcal{V}(P_i) = \bigcap_{j \neq i} h(P_i, P_j),$$

unde  $h(P_i, P_j)$  este semiplanul determinat de mediatoarea segmentului  $[P_i P_j]$  care conține punctul  $P_i$ . În particular: fiecare celulă este o mulțime convexă.

Aplicabilitate: algoritm (lent, complexitate-timp  $O(n^2 \log n)$ ) de determinare a diagramei Voronoi.

- ▶ Dacă toate punctele sunt coliniare, atunci diagrama Voronoi asociată  $\text{Vor}(\mathcal{P})$  conține  $n - 1$  *drepte paralele* între ele (în particular, pentru  $n \geq 3$ , ea nu este conexă). În caz contrar, diagrama este conexă, iar muchiile sale sunt fie *segmente*, fie *semidrepte* (cui corespund acestea?). Există o propoziție care caracterizează mediatoarele segmentelor  $[P_i P_j]$  care definesc muchii ale diagramei Voronoi.

# Structura unei diagrame Voronoi

- ▶ Fie  $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$  o mulțime de situri (puncte) din planul  $\mathbb{R}^2$ .
- ▶ Celula asociată unui punct este o intersecție de semiplane:

$$\mathcal{V}(P_i) = \bigcap_{j \neq i} h(P_i, P_j),$$

unde  $h(P_i, P_j)$  este semiplanul determinat de mediatoarea segmentului  $[P_i P_j]$  care conține punctul  $P_i$ . În particular: fiecare celulă este o mulțime convexă.

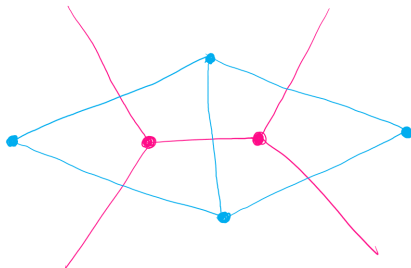
Aplicabilitate: algoritm (lent, complexitate-timp  $O(n^2 \log n)$ ) de determinare a diagramei Voronoi.

- ▶ Dacă toate punctele sunt coliniare, atunci diagrama Voronoi asociată  $\text{Vor}(\mathcal{P})$  conține  $n - 1$  *drepte paralele* între ele (în particular, pentru  $n \geq 3$ , ea nu este conexă). În caz contrar, diagrama este conexă, iar muchiile sale sunt fie *segmente*, fie *semidrepte* (cui corespund acestea?). Există o propoziție care caracterizează mediatoarele segmentelor  $[P_i P_j]$  care definesc muchii ale diagramei Voronoi.
- ▶ **Propoziție.** Fie o mulțime cu  $n$  situri. Atunci, pentru diagrama Voronoi asociată au loc inegalitățile

$$n_v \leq 2n - 5, \quad n_m \leq 3n - 6,$$

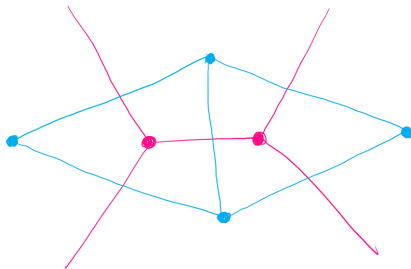
unde  $n_v$  este numărul de vârfuri ale diagramei și  $n_m$  este numărul de muchii al acesteia.

# Diagrame Voronoi și triangulări



► Construcție:

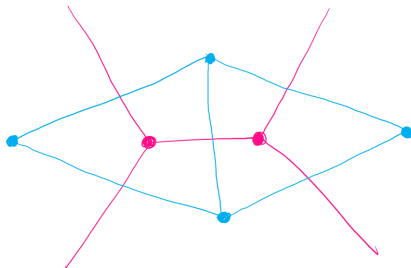
# Diagrame Voronoi și triangulări



## ► Construcție:

- Mulțime de puncte  $\mathcal{P}$  în planul  $\mathbb{R}^2 \implies$

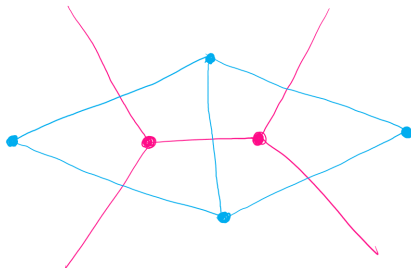
# Diagrame Voronoi și triangulări



## ► Construcție:

- Mulțime de puncte  $\mathcal{P}$  în planul  $\mathbb{R}^2 \implies$
- Diagrama Voronoi  $\text{Vor}(\mathcal{P}) \implies$

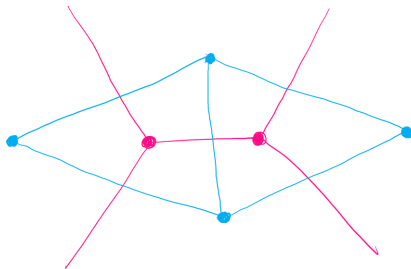
# Diagrame Voronoi și triangulări



## ► Construcție:

- Mulțime de puncte  $\mathcal{P}$  în planul  $\mathbb{R}^2 \implies$
- Diagrama Voronoi  $\text{Vor}(\mathcal{P}) \implies$
- Graful dual  $\mathcal{G}(\mathcal{P})$ . **Noduri:** fețele diagramei Voronoi (siturile). **Arce:** dacă celulele (fețele diagramei Voronoi corespunzătoare) au o muchie comună  $\implies$

# Diagrame Voronoi și triangulări



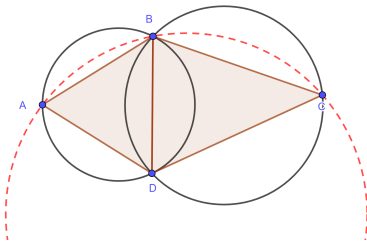
## ► Construcție:

- Mulțime de puncte  $\mathcal{P}$  în planul  $\mathbb{R}^2 \implies$
- Diagrama Voronoi  $\text{Vor}(\mathcal{P}) \implies$
- Graful dual  $\mathcal{G}(\mathcal{P})$ . **Noduri:** fețele diagramei Voronoi (siturile). **Arce:** dacă celulele (fețele diagramei Voronoi corespunzătoare) au o muchie comună  $\implies$
- Triangulare  $\mathcal{T}_{\mathcal{P}}$  (numită **triangulare Delaunay**)



# Legătura cu triangulările legale / unghiular optime

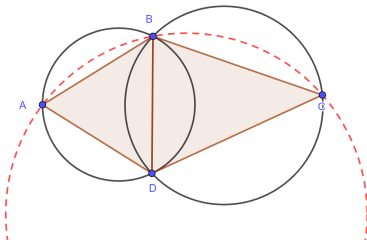
- **Propoziție.** Fie  $\mathcal{T}$  o triangulare a lui  $\mathcal{P}$ . Atunci  $\mathcal{T}$  este o triangulare Delaunay dacă și numai dacă pentru orice triunghi din  $\mathcal{T}$  cercul circumscris nu conține în interiorul său niciun punct al lui  $\mathcal{P}$ .



Pentru punctele  $A, B, C, D$  din figură, triangularea construită este o triangulare Delaunay. În schimb, triunghiul  $\triangle ABC$  nu poate participa la realizarea unei triangulări Delaunay, deoarece  $D$  este în interiorul cercului circumscris acestuia.

# Legătura cu triangulările legale / unghiular optime

- **Propoziție.** Fie  $\mathcal{T}$  o triangulare a lui  $\mathcal{P}$ . Atunci  $\mathcal{T}$  este o triangulare Delaunay dacă și numai dacă pentru orice triunghi din  $\mathcal{T}$  cercul circumscris nu conține în interiorul său niciun punct al lui  $\mathcal{P}$ .

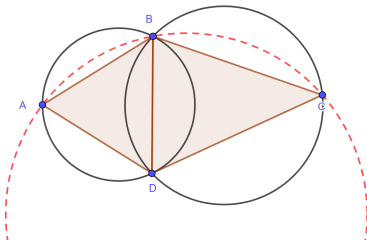


Pentru punctele  $A, B, C, D$  din figură, triangularea construită este o triangulare Delaunay. În schimb, triunghiul  $\triangle ABC$  nu poate participa la realizarea unei triangulări Delaunay, deoarece  $D$  este în interiorul cercului circumscris acestuia.

- **Teoremă.** O triangulare este legală dacă și numai dacă este o triangulare Delaunay.

# Legătura cu triangulările legale / unghiular optime

- **Propoziție.** Fie  $\mathcal{T}$  o triangulare a lui  $\mathcal{P}$ . Atunci  $\mathcal{T}$  este o triangulare Delaunay dacă și numai dacă pentru orice triunghi din  $\mathcal{T}$  cercul circumscris nu conține în interiorul său niciun punct al lui  $\mathcal{P}$ .



Pentru punctele  $A, B, C, D$  din figură, triangularea construită este o triangulare Delaunay. În schimb, triunghiul  $\triangle ABC$  nu poate participa la realizarea unei triangulări Delaunay, deoarece  $D$  este în interiorul cercului circumscris acestuia.

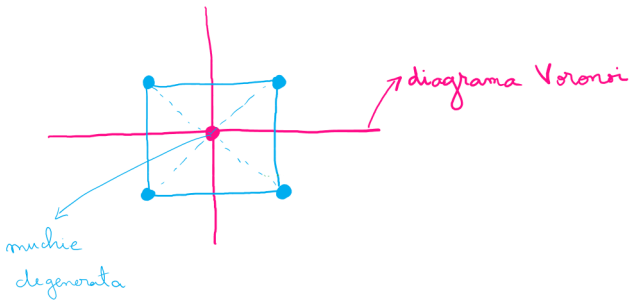
- **Teoremă.** O triangulare este legală dacă și numai dacă este o triangulare Delaunay.
- **Teoremă.** Orice triangulare unghiular optimă este o triangulare Delaunay. Pentru mulțimile în poziție generală (nu există patru puncte conciclice), orice triangulare Delaunay maximizează cel mai mic unghi, comparativ cu toate triangulările lui  $\mathcal{P}$ .

## Legătura cu triangulările legale / unghiular optime - cazul unui pătrat (patrulater inscriptibil)

**Întrebare:** Cum “funcționează” această construcție când punctele din  $\mathcal{P}$  sunt (de exemplu) vârfurile unui pătrat?

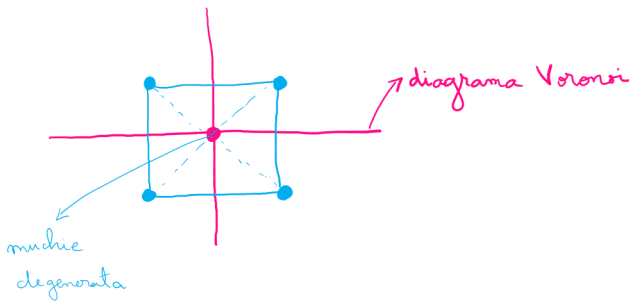
## Legătura cu triangulările legale / unghiular optime - cazul unui pătrat (patrulater inscriptibil)

**Întrebare:** Cum “funcționează” această construcție când punctele din  $\mathcal{P}$  sunt (de exemplu) vârfurile unui pătrat?



## Legătura cu triangulările legale / unghiular optime - cazul unui pătrat (patrulater inscriptibil)

**Întrebare:** Cum “funcționează” această construcție când punctele din  $\mathcal{P}$  sunt (de exemplu) vârfurile unui pătrat?



Oricare dintre cele două diagonale determină o triangulare Delaunay (legală), deoarece vârfurile corespunzătoare sunt “separate” de muchia degenerată.

# Algoritmul lui Fortune [1987]

- Metodă clasică (și eficientă) de determinare a diagramei Voronoi pentru o mulțime de puncte din planul  $\mathbb{R}^2$ . Complexitate:  $O(n \log n)$ .

Detalii:

<http://www.ams.org/samplings/feature-column/fcarc-voronoi>.

Suport vizual disponibil.

# Algoritmul lui Fortune [1987]

- ▶ Metodă clasică (și eficientă) de determinare a diagramei Voronoi pentru o mulțime de puncte din planul  $\mathbb{R}^2$ . Complexitate:  $O(n \log n)$ .  
Detalii:  
<http://www.ams.org/samplings/feature-column/fcarc-voronoi>.  
Suport vizual disponibil.
- ▶ **Principiu (paradigmă):** sweep line / dreaptă de baleiere.

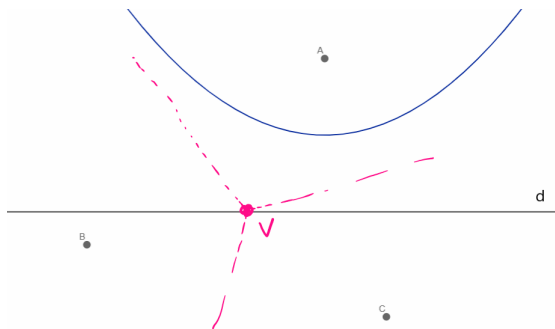


## Adaptarea paradigmei drepte de baleiere - problematizare

**Inconvenient:** la întâlnirea unui vârf al diagramei, dreapta de baleiere nu a întâlnit încă toate siturile (puncte din  $\mathcal{P}$ ) care determină acest vârf! În figură, la trecerea prin vârful  $V$  al diagramei Voronoi, dreapta de baleiere  $d$  nu a întâlnit încă toate siturile care determină  $V$ . Altfel spus,  $V$  ar trebui detectat înainte de a trece prin  $B$  și  $C$ .

# Adaptarea paradigmei drepte de baleiere - problematizare

**Inconvenient:** la întâlnirea unui vârf al diagramei, dreapta de baleiere nu a întâlnit încă toate siturile (puncte din  $\mathcal{P}$ ) care determină acest vârf! În figură, la trecerea prin vârful  $V$  al diagramei Voronoi, dreapta de baleiere  $d$  nu a întâlnit încă toate siturile care determină  $V$ . Altfel spus,  $V$  ar trebui detectat înainte de a trece prin  $B$  și  $C$ .

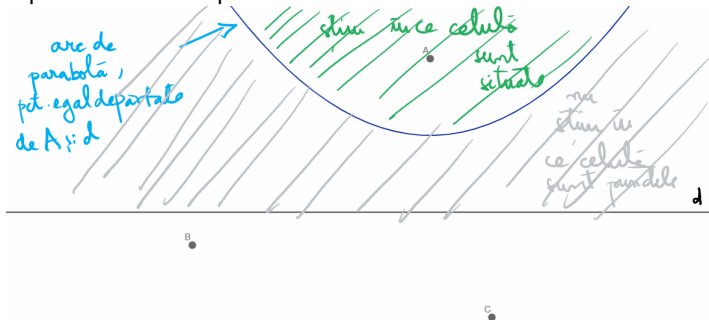


## Adaptarea paradigmei drepte de baleiere

**Adaptare:** nu reținem informația legată de intersecția dintre dreapta de baleiere și diagramă, ci doar informația legată de partea diagramei care nu mai poate fi influențată de punctele situate de dincolo de dreapta de baleiere. Acest lucru se realizează asociind fiecărui sit câte o parabolă - mulțimea punctelor egal depărtate de sit și de dreapta de baleiere. Pe parcurs (i) aceste parabole își schimbă forma, (ii) pot să dispară / să fie “descompuse” în arce de parabolă.

## Adaptarea paradigmei drepte de baleiere

**Adaptare:** nu reținem informația legată de intersecția dintre dreapta de baleiere și diagramă, ci doar informația legată de partea diagramei care nu mai poate fi influențată de punctele situate de dincolo de dreapta de baleiere. Acest lucru se realizează asociind fiecărui sit câte o parabolă - mulțimea punctelor egal depărtate de sit și de dreapta de baleiere. Pe parcurs (i) aceste parabole își schimbă forma, (ii) pot să dispară / să fie "descompuse" în arce de parabolă.

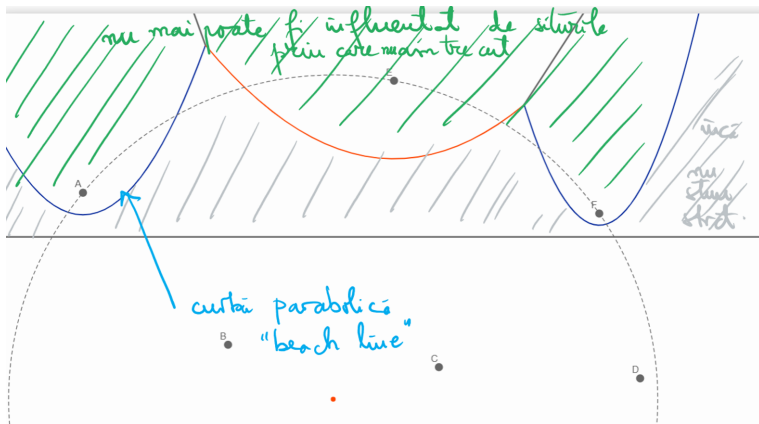


# Adaptarea paradigmei drepte de baleiere - curba parabolică

Din punct de vedere practic, apare o reuniune de arce de parabolă (**curbă parabolică**), ceea ce este situat deasupra acestei curbe nu mai poate fi influențat de evenimentele nedetectate.

# Adaptarea paradigmei drepte de baleiere - curba parabolică

Din punct de vedere practic, apare o reuniune de arce de parabolă (**curbă parabolică**), ceea ce este situat deasupra acestei curbe nu mai poate fi influențat de evenimentele nedetectate.



# Despre curba parabolică (I)

- ▶ **Curba parabolică** (*beach line*):

# Despre curba parabolică (I)

## ► Curba parabolică (*beach line*):

- Este o reuniune de arce de parabolă.
- Un punct de pe curba parabolică este egal depărtat de situl care determină arcul de parabolă și dreapta de baleiere. Presupunem că dreapta de baleiere  $d$  are ecuația  $y = 0$ , iar situl  $P_i$ , situat deasupra lui  $d$  (adică  $y_i > 0$ ) are coordonatele  $(x_i, y_i)$ . Locul geometric al punctelor egal depărtate de  $d$  și  $P_i$  are ecuația

$$y = \frac{1}{2y_i}x^2 - \frac{x_i}{y_i}x + \frac{x_i^2 + y_i^2}{2y_i}.$$



# Despre curba parabolică (I)

## ► Curba parabolică (*beach line*):

- Este o reuniune de arce de parabolă.
- Un punct de pe curba parabolică este egal depărtat de situl care determină arcul de parabolă și dreapta de baleiere. Presupunem că dreapta de baleiere  $d$  are ecuația  $y = 0$ , iar situl  $P_i$ , situat deasupra lui  $d$  (adică  $y_i > 0$ ) are coordonatele  $(x_i, y_i)$ . Locul geometric al punctelor egal depărtate de  $d$  și  $P_i$  are ecuația

$$y = \frac{1}{2y_i}x^2 - \frac{x_i}{y_i}x + \frac{x_i^2 + y_i^2}{2y_i}.$$

- Punctele de racord ale arcelor de parabolă aparțin muchiilor diagramei Voronoi.

# Despre curba parabolică (I)

## ► Curba parabolică (*beach line*):

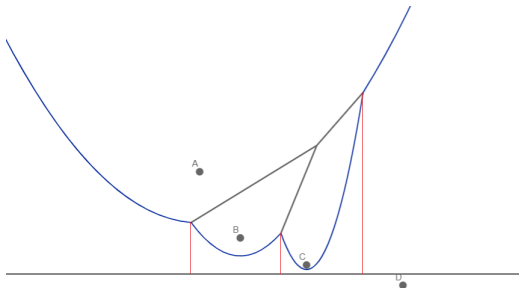
- Este o reuniune de arce de parabolă.
- Un punct de pe curba parabolică este egal depărtat de situl care determină arcul de parabolă și dreapta de baleiere. Presupunem că dreapta de baleiere  $d$  are ecuația  $y = 0$ , iar situl  $P_i$ , situat deasupra lui  $d$  (adică  $y_i > 0$ ) are coordonatele  $(x_i, y_i)$ . Locul geometric al punctelor egal depărtate de  $d$  și  $P_i$  are ecuația

$$y = \frac{1}{2y_i}x^2 - \frac{x_i}{y_i}x + \frac{x_i^2 + y_i^2}{2y_i}.$$

- Punctele de racord ale arcelor de parabolă aparțin muchiilor diagramei Voronoi.
- Curba parabolică este  $x$ -monotonă, adică orice dreaptă verticală o intersectează *exact* într-un punct (la ce folosește această proprietate?).

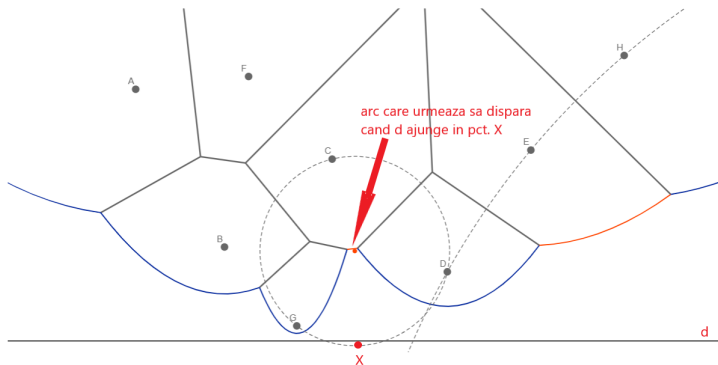
## Despre curba parabolică (II) - modificarea curbei parabolice

**Site event / eveniment de tip locație.** (a) La trecerea printr-un sit apare un arc de parabolă (care “la început” este degenerat) și, reciproc, apariția unui nou arc este posibilă doar la trecerea printr-un sit. (b) Cum stabilim algoritmic / numeric ce arc de parabolă este “splitat”? - v. figura de mai jos. Pentru o poziție fixată a dreptei de baleiere pot fi determinate coordonatele proiecțiilor punctelor de racord dintre arcele de parabolă pe dreapta de baleiere și, în particular, poate fi găsit arcul de parabolă situat deasupra unui eveniment. (c) La un moment fixat, curba parabolică are maxim  $(2n - 1)$  arce.



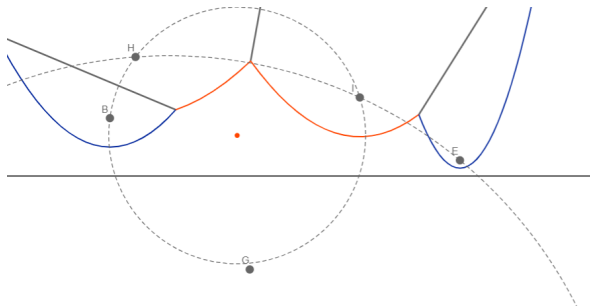
## Despre curba parabolică (II) - modificarea curbei parabolice

**Circle event / eveniment de tip cerc.** (a) La întâlnirea punctului inferior al unui cerc care trece prin cel puțin trei situri și este tangent la dreapta de baleiere dispare un arc de parabolă și, reciproc, arcele de parabolă dispar doar la acest tip de evenimente. În figură este cercul dat de punctele  $G, C, D$ . Arcul care dispare este cel corespunzător punctului  $C$ . Un astfel de eveniment este asociat unui vârf al diagramei Voronoi.



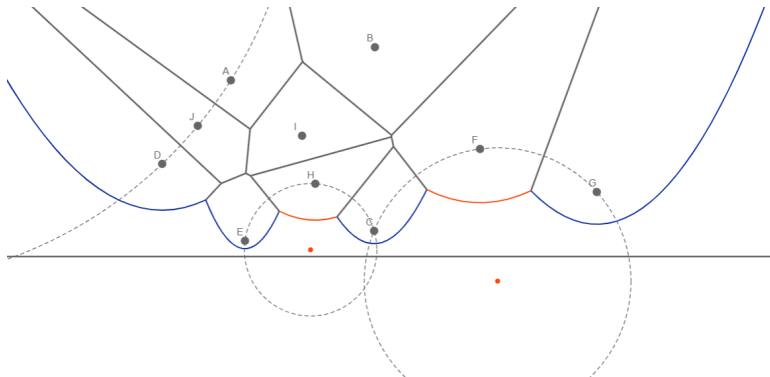
# Despre curba parabolică (II) - modificarea curbei parabolice

**Circle event / eveniment de tip cerc.** (b) Un eveniment de tip cerc este dat de trei arce de parabolă consecutive de pe curba parabolică, deci trebuie testate toate tripletele consecutive de arce, pe măsură ce ele apar. În figură, evenimentele de tip cerc “active” sunt  $\mathcal{C}_{(HIE)}$  și  $\mathcal{C}_{(BHI)}$ .



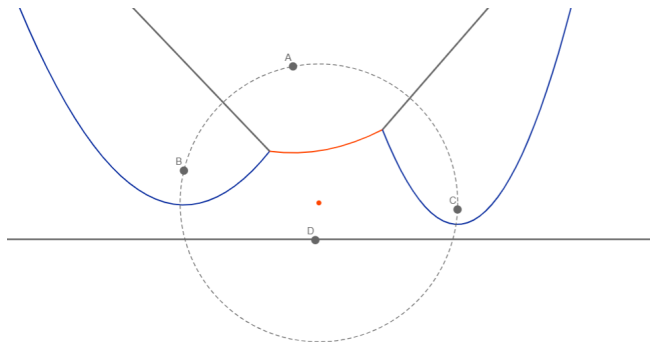
# Despre curba parabolică (II) - modificarea curbei parabolice

**Circle event / eveniment de tip cerc.** (c) Există triplete de arce consecutive (muchii ale diagramei Voronoi) pentru care muchiile nu se întâlnesc. În figură, de exemplu, această afirmație se aplică arcelor determinate de punctele  $D$ ,  $E$  și  $H$ .



# Despre curba parabolică (II) - modificarea curbei parabolice

**Circle event / eveniment de tip cerc.** (d) Unele evenimente de tip cerc detectate nu au loc. În figură, evenimentul de tip cerc corespunzător  $C_{(ABC)}$  nu va avea loc. Arcul de parabolă corespunzător va fi “splitat” la evenimentul de tip locație  $D$ .



# Adaptarea paradigmei drepte de baleiere - formalizare

- ▶ **Statut:** structura curbei parabolice (succesiunea de arce de parabolă - se subînțelege că este reținută ordinea arcelor); este modificat de două tipuri de evenimente



# Adaptarea paradigmei dreptei de baleiere - formalizare

- ▶ **Statut:** structura curbei parabolice (succesiunea de arce de parabolă - se subînțelege că este reținută ordinea arcelor); este modificat de două tipuri de evenimente
- ▶ **Evenimente:**
  - ▶ site event / eveniment de tip locație: întâlnirea unui sit, adică a unui punct din mulțimea  $\mathcal{P}$  (apare un arc de parabolă)
  - ▶ circle event / eveniment de tip cerc: întâlnirea unui “punct inferior” al unui cerc care trece prin cel puțin trei situri, tangent la dreapta de baleiere (dispare un arc de parabolă) - **vârf al diagramei Voronoi**

## Structuri de date utilizate

- **Diagrama Voronoi:** *doubly connected edge list (DCEL)*  $\mathcal{D}$  (ulterior).

## Structuri de date utilizate

- ▶ **Diagrama Voronoi:** *doubly connected edge list (DCEL)*  $\mathcal{D}$  (ulterior).
- ▶ **Evenimente:** coadă de priorități / evenimente  $\mathcal{Q}$  și arbore de căutare binar echilibrat. Sunt reținute evenimentele de tip sit, precum și evenimentele de tip cerc - detectate pe parcursul algoritmului.

## Structuri de date utilizate

- ▶ **Diagrama Voronoi:** *doubly connected edge list (DCEL)*  $\mathcal{D}$  (ulterior).
- ▶ **Evenimente:** coadă de priorități / evenimente  $\mathcal{Q}$  și arbore de căutare binar echilibrat. Sunt reținute evenimentele de tip sit, precum și evenimentele de tip cerc - detectate pe parcursul algoritmului.
- ▶ **Statut:** Structura curbei parabolice - arbore de căutare binar echilibrat  $\mathcal{T}$ .

## Structuri de date utilizate

- ▶ **Diagrama Voronoi:** *doubly connected edge list (DCEL)*  $\mathcal{D}$  (ulterior).
- ▶ **Evenimente:** coadă de priorități / evenimente  $\mathcal{Q}$  și arbore de căutare binar echilibrat. Sunt reținute evenimentele de tip sit, precum și evenimentele de tip cerc - detectate pe parcursul algoritmului.
- ▶ **Statut:** Structura curbei parabolice - arbore de căutare binar echilibrat  $\mathcal{T}$ .
  - ▶ **pe frunze:** siturile care au arce active, la ajungerea într-un sit se inserează un arc, iar la un eveniment de tip cerc se șterge un arc;

## Structuri de date utilizate

- ▶ **Diagrama Voronoi:** *doubly connected edge list (DCEL)*  $\mathcal{D}$  (ulterior).
- ▶ **Evenimente:** coadă de priorități / evenimente  $\mathcal{Q}$  și arbore de căutare binar echilibrat. Sunt reținute evenimentele de tip sit, precum și evenimentele de tip cerc - detectate pe parcursul algoritmului.
- ▶ **Statut:** Structura curbei parabolice - arbore de căutare binar echilibrat  $\mathcal{T}$ .
  - ▶ **pe frunze:** siturile care au arce active, la ajungerea într-un sit se inserează un arc, iar la un eveniment de tip cerc se șterge un arc;
  - ▶ **în nodurile interne:** punctele de racord ale arcelor de parabolă (memorate simbolic) - corespund muchiilor diagramei Voronoi

## Structuri de date utilizate

- ▶ **Diagrama Voronoi:** *doubly connected edge list (DCEL)*  $\mathcal{D}$  (ulterior).
- ▶ **Evenimente:** coadă de priorități / evenimente  $\mathcal{Q}$  și arbore de căutare binar echilibrat. Sunt reținute evenimentele de tip sit, precum și evenimentele de tip cerc - detectate pe parcursul algoritmului.
- ▶ **Statut:** Structura curbei parabolice - arbore de căutare binar echilibrat  $\mathcal{T}$ .
  - ▶ **pe frunze:** siturile care au arce active, la ajungerea într-un sit se inserează un arc, iar la un eveniment de tip cerc se șterge un arc;
  - ▶ **în nodurile interne:** punctele de racord ale arcelor de parabolă (memorate simbolic) - corespund muchiilor diagramei Voronoi
  - ▶ pointeri (eventual nuli) de la frunze către evenimentele de tip cerc; de la nodurile interne către muchiile diagramei Voronoi

# Structuri de date utilizate

- ▶ **Diagrama Voronoi:** *doubly connected edge list (DCEL)*  $\mathcal{D}$  (ulterior).
- ▶ **Evenimente:** coadă de priorități / evenimente  $\mathcal{Q}$  și arbore de căutare binar echilibrat. Sunt reținute evenimentele de tip sit, precum și evenimentele de tip cerc - detectate pe parcursul algoritmului.
- ▶ **Statut:** Structura curbei parabolice - arbore de căutare binar echilibrat  $\mathcal{T}$ .
  - ▶ **pe frunze:** siturile care au arce active, la ajungerea într-un sit se inserează un arc, iar la un eveniment de tip cerc se șterge un arc;
  - ▶ **în nodurile interne:** punctele de racord ale arcelor de parabolă (memorate simbolic) - corespund muchiilor diagramei Voronoi
  - ▶ pointeri (eventual nuli) de la frunze către evenimentele de tip cerc; de la nodurile interne către muchiile diagramei Voronoi
- ▶ **Analiză preliminară a complexității:** (i) Câte evenimente sunt în total? (ii) Care este complexitatea-timp pentru a actualiza coada de evenimente la un eveniment de tip cerc? (iii) Care este complexitatea-timp (totală) pentru a actualiza coada de evenimente? (iv) Întrebări similare pentru statut.



# Algoritmul

**Input.** O mulțime de situri  $\mathcal{P} = \{p_1, \dots, p_n\}$  de situri în plan.

**Output.** Diagrama Voronoi  $\text{Vor}(\mathcal{P})$  în interiorul unui *bounding box*, descrisă printr-o listă dublu înlănțuită  $\mathcal{D}$ .

1. Inițializări: coada de evenimente  $\mathcal{Q} \leftarrow \mathcal{P}$  (preprocesare: ordonare după  $y$ ), statut (arbore balansat)  $\mathcal{T} \leftarrow \emptyset$ ; listă dublu înlănțuită  $\mathcal{D} \leftarrow \emptyset$ .

# Algoritmul

**Input.** O mulțime de situri  $\mathcal{P} = \{p_1, \dots, p_n\}$  de situri în plan.

**Output.** Diagrama Voronoi  $\text{Vor}(\mathcal{P})$  în interiorul unui *bounding box*, descrisă printr-o listă dublu înlănțuită  $\mathcal{D}$ .

1. Inițializări: coada de evenimente  $\mathcal{Q} \leftarrow \mathcal{P}$  (preprocesare: ordonare după  $y$ ), statut (arbore balansat)  $\mathcal{T} \leftarrow \emptyset$ ; listă dublu înlănțuită  $\mathcal{D} \leftarrow \emptyset$ .
2. **while**  $\mathcal{Q} \neq \emptyset$

# Algoritmul

**Input.** O mulțime de situri  $\mathcal{P} = \{p_1, \dots, p_n\}$  de situri în plan.

**Output.** Diagrama Voronoi  $\text{Vor}(\mathcal{P})$  în interiorul unui *bounding box*, descrisă printr-o listă dublu înlănțuită  $\mathcal{D}$ .

1. Inițializări: coada de evenimente  $\mathcal{Q} \leftarrow \mathcal{P}$  (preprocesare: ordonare după  $y$ ), statut (arbore balansat)  $\mathcal{T} \leftarrow \emptyset$ ; listă dublu înlănțuită  $\mathcal{D} \leftarrow \emptyset$ .
2. **while**  $\mathcal{Q} \neq \emptyset$
3.     **do** elimină evenimentul cu cel mai mare  $y$  din  $\mathcal{Q}$

# Algoritmul

**Input.** O mulțime de situri  $\mathcal{P} = \{p_1, \dots, p_n\}$  de situri în plan.

**Output.** Diagrama Voronoi  $\text{Vor}(\mathcal{P})$  în interiorul unui *bounding box*, descrisă printr-o listă dublu înlănțuită  $\mathcal{D}$ .

1. Inițializări: coada de evenimente  $\mathcal{Q} \leftarrow \mathcal{P}$  (preprocesare: ordonare după  $y$ ), statut (arbore balansat)  $\mathcal{T} \leftarrow \emptyset$ ; listă dublu înlănțuită  $\mathcal{D} \leftarrow \emptyset$ .
2. **while**  $\mathcal{Q} \neq \emptyset$
3.     **do** elimină evenimentul cu cel mai mare  $y$  din  $\mathcal{Q}$
4.         **if** evenimentul **ev** este un eveniment de tip sit

# Algoritmul

**Input.** O mulțime de situri  $\mathcal{P} = \{p_1, \dots, p_n\}$  de situri în plan.

**Output.** Diagrama Voronoi  $\text{Vor}(\mathcal{P})$  în interiorul unui *bounding box*, descrisă printr-o listă dublu înlănțuită  $\mathcal{D}$ .

1. Inițializări: coada de evenimente  $\mathcal{Q} \leftarrow \mathcal{P}$  (preprocesare: ordonare după  $y$ ), statut (arbore balansat)  $\mathcal{T} \leftarrow \emptyset$ ; listă dublu înlănțuită  $\mathcal{D} \leftarrow \emptyset$ .
2. **while**  $\mathcal{Q} \neq \emptyset$
3.     **do** elimină evenimentul cu cel mai mare  $y$  din  $\mathcal{Q}$
4.         **if** evenimentul **ev** este un eveniment de tip sit
5.             **then**  $\text{PROCESSEVSIT}(p_i)$ , cu  $p_i = \text{ev}$

# Algoritmul

**Input.** O mulțime de situri  $\mathcal{P} = \{p_1, \dots, p_n\}$  de situri în plan.

**Output.** Diagrama Voronoi  $\text{Vor}(\mathcal{P})$  în interiorul unui *bounding box*, descrisă printr-o listă dublu înlănțuită  $\mathcal{D}$ .

1. Inițializări: coada de evenimente  $\mathcal{Q} \leftarrow \mathcal{P}$  (preprocesare: ordonare după  $y$ ), statut (arbore balansat)  $\mathcal{T} \leftarrow \emptyset$ ; listă dublu înlănțuită  $\mathcal{D} \leftarrow \emptyset$ .
2. **while**  $\mathcal{Q} \neq \emptyset$
3.     **do** elimină evenimentul cu cel mai mare  $y$  din  $\mathcal{Q}$
4.         **if** evenimentul **ev** este un eveniment de tip sit
5.             **then**  $\text{PROCESSEVSIT}(p_i)$ , cu  $p_i = \mathbf{ev}$
6.             **else**  $\text{PROCESSEVCERC}(\gamma)$ , cu  $\gamma = \text{arc}(\mathbf{ev}) \in \mathcal{T}$

# Algoritmul

**Input.** O mulțime de situri  $\mathcal{P} = \{p_1, \dots, p_n\}$  de situri în plan.

**Output.** Diagrama Voronoi  $\text{Vor}(\mathcal{P})$  în interiorul unui *bounding box*, descrisă printr-o listă dublu înlănțuită  $\mathcal{D}$ .

1. Inițializări: coada de evenimente  $\mathcal{Q} \leftarrow \mathcal{P}$  (preprocesare: ordonare după  $y$ ), statut (arbore balansat)  $\mathcal{T} \leftarrow \emptyset$ ; listă dublu înlănțuită  $\mathcal{D} \leftarrow \emptyset$ .
2. **while**  $\mathcal{Q} \neq \emptyset$
3.     **do** elimină evenimentul cu cel mai mare  $y$  din  $\mathcal{Q}$
4.         **if** evenimentul **ev** este un eveniment de tip sit
5.             **then**  $\text{PROCESSEVSIT}(p_i)$ , cu  $p_i = \mathbf{ev}$
6.             **else**  $\text{PROCESSEVCERC}(\gamma)$ , cu  $\gamma = \text{arc}(\mathbf{ev}) \in \mathcal{T}$
7. Nodurile interne încă prezente în  $\mathcal{T}$  corespund semidreptelor diagramei Voronoi. Consideră un *bounding box* care conține toate vârfurile diagramei Voronoi în interiorul său și leagă semidreptele de acest *bounding box*, prin actualizarea corespunzătoare a lui  $\mathcal{D}$ .

# Algoritmul

**Input.** O mulțime de situri  $\mathcal{P} = \{p_1, \dots, p_n\}$  de situri în plan.

**Output.** Diagrama Voronoi  $\text{Vor}(\mathcal{P})$  în interiorul unui *bounding box*, descrisă printr-o listă dublu înălțuită  $\mathcal{D}$ .

1. Inițializări: coada de evenimente  $\mathcal{Q} \leftarrow \mathcal{P}$  (preprocesare: ordonare după  $y$ ), statut (arbore balansat)  $\mathcal{T} \leftarrow \emptyset$ ; listă dublu înălțuită  $\mathcal{D} \leftarrow \emptyset$ .
2. **while**  $\mathcal{Q} \neq \emptyset$
3.     **do** elimină evenimentul cu cel mai mare  $y$  din  $\mathcal{Q}$
4.         **if** evenimentul **ev** este un eveniment de tip sit
5.             **then**  $\text{PROCESSEVSIT}(p_i)$ , cu  $p_i = \mathbf{ev}$
6.             **else**  $\text{PROCESSEVCERC}(\gamma)$ , cu  $\gamma = \text{arc}(\mathbf{ev}) \in \mathcal{T}$
7. Nodurile interne încă prezente în  $\mathcal{T}$  corespund semidreptelor diagramei Voronoi. Consideră un *bounding box* care conține toate vârfurile diagramei Voronoi în interiorul său și leagă semidreptele de acest *bounding box*, prin actualizarea corespunzătoare a lui  $\mathcal{D}$ .
8. Traversează muchiile pentru a adăuga celulele diagramei și pointeri corespunzători.



## Procedura PROCESSEvSIT ( $p_i$ )

1. Dacă  $\mathcal{T}$  este vidă, inserează  $p_i$  și revine, dacă nu continuă cu 2.–5.

## Procedura PROCESSEvSIT ( $p_i$ )

1. Dacă  $\mathcal{T}$  este vidă, inserează  $p_i$  și revine, dacă nu continuă cu 2.—5.
2. Caută în  $\mathcal{T}$  arcul  $\alpha$  situat deasupra lui  $p_i$ . Dacă frunza reprezentând  $\alpha$  are un pointer către un eveniment de tip cerc **ev** din  $\mathcal{Q}$ , atunci **ev** este o alarmă falsă și trebuie șters.

## Procedura PROCESSEvSIT ( $p_i$ )

1. Dacă  $\mathcal{T}$  este vidă, inserează  $p_i$  și revine, dacă nu continuă cu 2.—5.
2. Caută în  $\mathcal{T}$  arcul  $\alpha$  situat deasupra lui  $p_i$ . Dacă frunza reprezentând  $\alpha$  are un pointer către un eveniment de tip cerc **ev** din  $\mathcal{Q}$ , atunci **ev** este o alarmă falsă și trebuie șters.
3. Înlocuiește frunza lui  $\mathcal{T}$  care reprezintă  $\alpha$  cu un subarbore cu trei frunze: cea din mijloc reține situl  $p_i$  și celelalte două situl  $p_j$  asociat lui  $\alpha$ . Memorează perechile reprezentând punctele de racord în două noduri interne. Efectuează rebalansări în  $\mathcal{T}$ , dacă este necesar.

## Procedura PROCESSEvSIT ( $p_i$ )

1. Dacă  $\mathcal{T}$  este vidă, inserează  $p_i$  și revine, dacă nu continuă cu 2.—5.
2. Caută în  $\mathcal{T}$  arcul  $\alpha$  situat deasupra lui  $p_i$ . Dacă frunza reprezentând  $\alpha$  are un pointer către un eveniment de tip cerc **ev** din  $\mathcal{Q}$ , atunci **ev** este o alarmă falsă și trebuie șters.
3. Înlocuiește frunza lui  $\mathcal{T}$  care reprezintă  $\alpha$  cu un subarbore cu trei frunze: cea din mijloc reține situl  $p_i$  și celelalte două situl  $p_j$  asociat lui  $\alpha$ . Memorează perechile reprezentând punctele de racord în două noduri interne. Efectuează rebalansări în  $\mathcal{T}$ , dacă este necesar.
4. Generează noi înregistrări de tip semi-muchie în structura diagramei Voronoi ( $\mathcal{D}$ ), pentru muchiile care separă celulele  $V(p_i)$  și  $V(p_j)$ , corespunzând celor două noi puncte de racord.

## Procedura PROCESSEvSIT ( $p_i$ )

1. Dacă  $\mathcal{T}$  este vidă, inserează  $p_i$  și revine, dacă nu continuă cu 2.—5.
2. Caută în  $\mathcal{T}$  arcul  $\alpha$  situat deasupra lui  $p_i$ . Dacă frunza reprezentând  $\alpha$  are un pointer către un eveniment de tip cerc **ev** din  $\mathcal{Q}$ , atunci **ev** este o alarmă falsă și trebuie șters.
3. Înlocuiește frunza lui  $\mathcal{T}$  care reprezintă  $\alpha$  cu un subarbore cu trei frunze: cea din mijloc reține situl  $p_i$  și celelalte două situl  $p_j$  asociat lui  $\alpha$ . Memorează perechile reprezentând punctele de racord în două noduri interne. Efectuează rebalansări în  $\mathcal{T}$ , dacă este necesar.
4. Generează noi înregistrări de tip semi-muchie în structura diagramei Voronoi ( $\mathcal{D}$ ), pentru muchiile care separă celulele  $V(p_i)$  și  $V(p_j)$ , corespunzând celor două noi puncte de racord.
5. Verifică tripletele de arce consecutive nou create, pentru a verifica dacă muchiile corespunzătoare punctelor de racord se întâlnesc. Dacă da, inserează evenimente de tip cerc în  $\mathcal{Q}$  și adaugă pointeri de la nodurile lui  $\mathcal{T}$  la evenimentele corespunzătoare din  $\mathcal{Q}$ .

# Procedura PROCESSEVCERC ( $\gamma$ )

1. Șterge frunza  $\gamma \in \mathcal{T}$  care corespunde arcului de cerc  $\alpha$  care dispare. Actualizează în nodurile interne perechile care corespund punctelor de racord. Efectuează rebalansări în  $\mathcal{T}$ , dacă este necesar. Șterge toate evenimentele de tip cerc care îi corespund lui  $\alpha$  (cu ajutorul pointerilor de la predecesorul și succesorul lui  $\gamma$  în  $\mathcal{T}$ ).

# Procedura PROCESSEVCERC ( $\gamma$ )

1. Șterge frunza  $\gamma \in \mathcal{T}$  care corespunde arcului de cerc  $\alpha$  care dispare. Actualizează în nodurile interne perechile care corespund punctelor de racord. Efectuează rebalansări în  $\mathcal{T}$ , dacă este necesar. Șterge toate evenimentele de tip cerc care îi corespund lui  $\alpha$  (cu ajutorul pointerilor de la predecesorul și succesorul lui  $\gamma$  în  $\mathcal{T}$ ).
2. Adaugă centrul cercului care determină evenimentul ca înregistrare de tip vârf în  $\mathcal{D}$ . Creează înregistrări de tip semi-muchie corespunzând noului punct de racord de pe linia parabolică și asignează pointeri corespunzători.

# Procedura PROCESSEVCERC ( $\gamma$ )

1. Șterge frunza  $\gamma \in \mathcal{T}$  care corespunde arcului de cerc  $\alpha$  care dispare. Actualizează în nodurile interne perechile care corespund punctelor de racord. Efectuează rebalansări în  $\mathcal{T}$ , dacă este necesar. Șterge toate evenimentele de tip cerc care îi corespund lui  $\alpha$  (cu ajutorul pointerilor de la predecesorul și succesorul lui  $\gamma$  în  $\mathcal{T}$ ).
2. Adaugă centrul cercului care determină evenimentul ca înregistrare de tip vârf în  $\mathcal{D}$ . Creează înregistrări de tip semi-muchie corespunzând noului punct de racord de pe linia parabolică și asignează pointeri corespunzători.
3. Verifică tripletele de arce consecutive nou create (care au foștii vecini ai lui  $\alpha$  în centru), pentru a verifica dacă muchiile corespunzătoare punctelor de racord se întâlnesc. Dacă da, inserează evenimente de tip cerc în  $\mathcal{Q}$  și adaugă pointeri de la nodurile lui  $\mathcal{T}$  la evenimentele corespunzătoare din  $\mathcal{Q}$ .



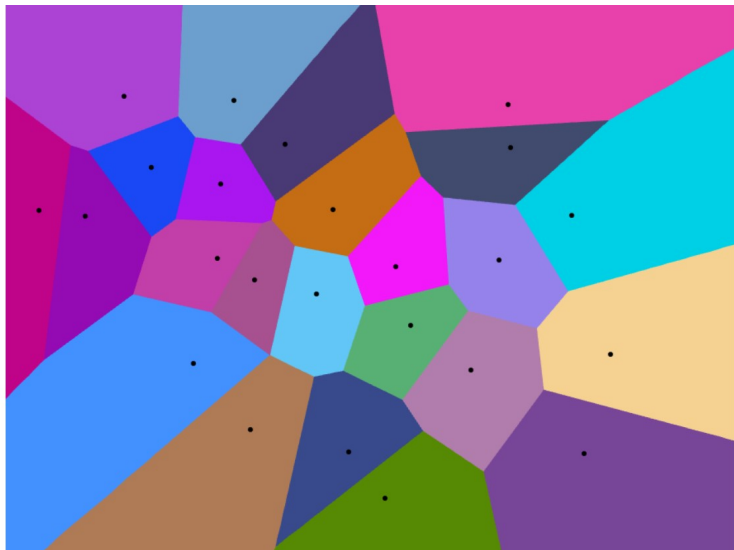
## Rezultate principale

- **Teoremă.** *Diagrama Voronoi a unei mulțimi de  $n$  situri poate fi determinată cu un algoritm bazat pe paradigma dreptei de baleiere având complexitate-timp  $O(n \log n)$  și complexitate-spațiu  $O(n)$ .*

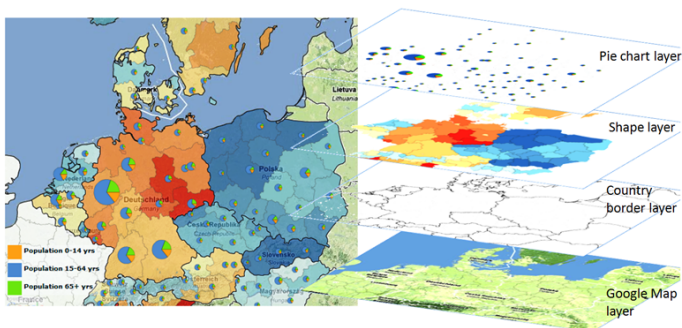
# Rezultate principale

- ▶ **Teoremă.** *Diagrama Voronoi a unei mulțimi de  $n$  situri poate fi determinată cu un algoritm bazat pe paradigma dreptei de baleiere având complexitate-timp  $O(n \log n)$  și complexitate-spațiu  $O(n)$ .*
- ▶ **Teoremă.** *Triangularea Delaunay a unei mulțimi de  $n$  situri poate fi determinată cu un algoritm bazat pe paradigma dreptei de baleiere având complexitate-timp  $O(n \log n)$  și complexitate-spațiu  $O(n)$ .*

# Motivație - reprezentarea diagramelor Voronoi



# Motivație - reprezentarea datelor geo-spațiale



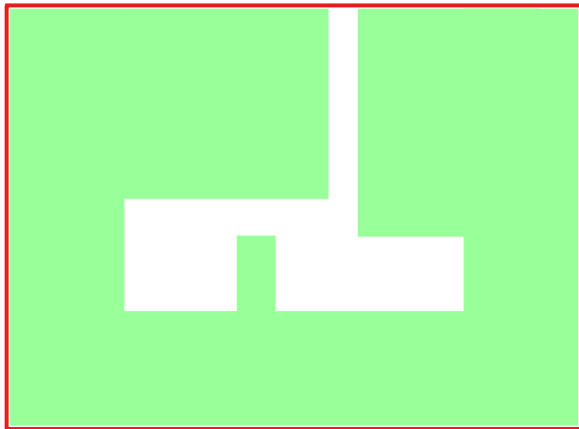
Sursa: <https://s-media-cache-ak0.pinimg.com/originals/37/90/86/37908600ab7db99c424c3bc6e1ddb740.jpg>

Ce structură de date este adecvată pentru a memora astfel de informații?

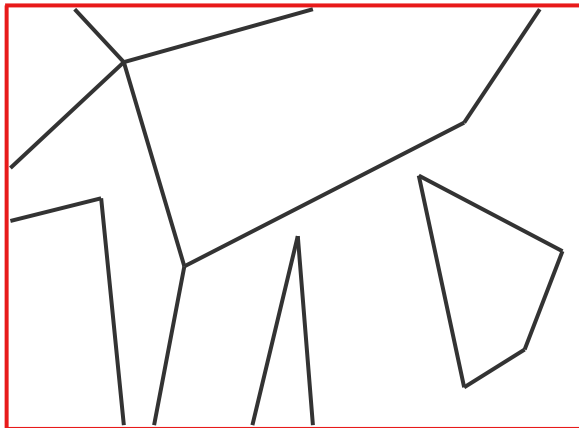
# Problematizare



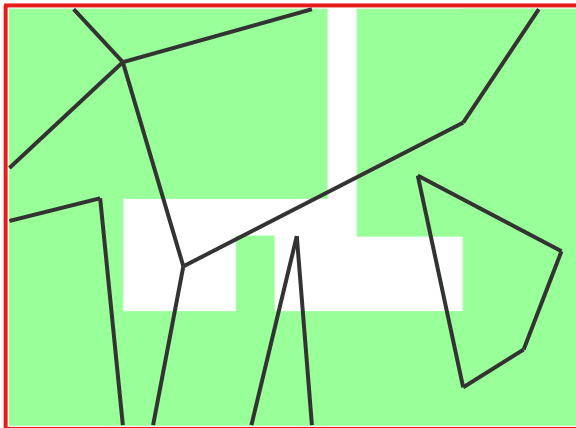
# Problematizare



# Problematizare



# Problematizare

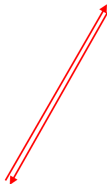




# Ideea cheie

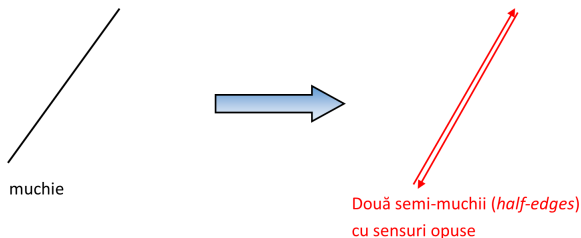


muchie



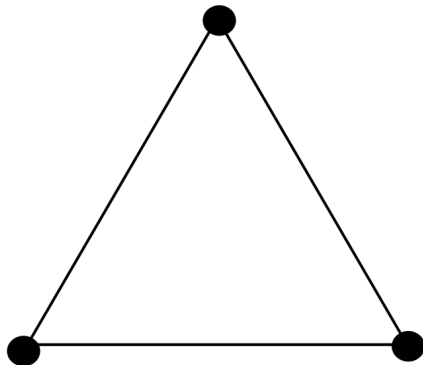
Două semi-muchii (*half-edges*)  
cu sensuri opuse

# Ideea cheie

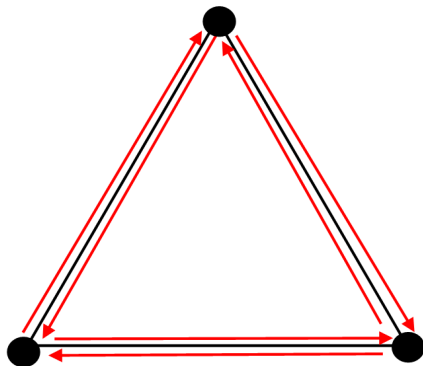


- Ideea fundamentală este de a introduce conceptul de semi-muchie (muchie orientată), cf. "*half-edge*".

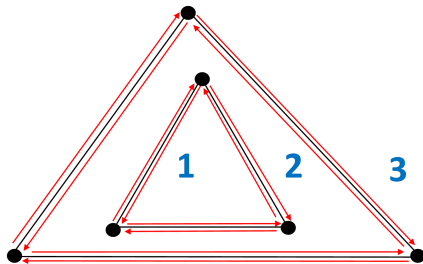
# Exemplu



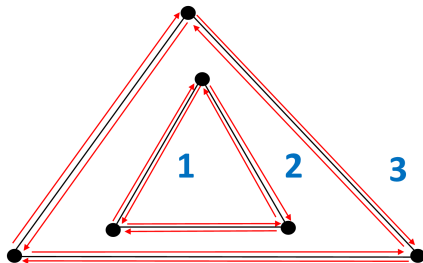
## Exemplu



## Exemplu

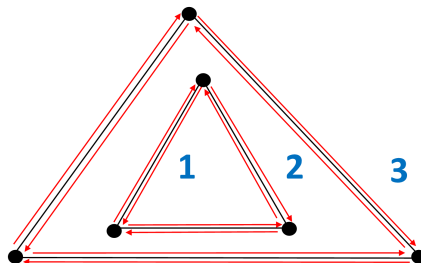


# Exemplu



Subdiviziunea din figură are 6 vârfuri, 12 semi-muchii, 3 fețe.

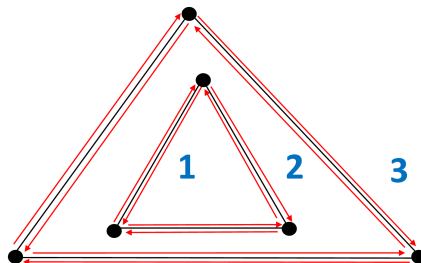
# Exemplu



Subdiviziunea din figură are 6 vârfuri, 12 semi-muchii, 3 fețe.

Semi-muchiile delimitează frontiere ale fețelor, fața delimitată fiind la stânga.

# Exemplu



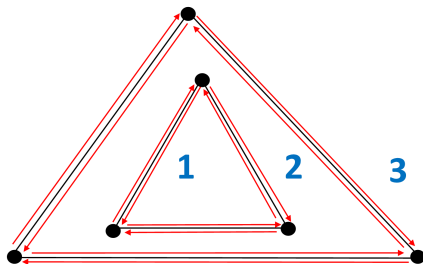
Subdiviziunea din figură are 6 vârfuri, 12 semi-muchii, 3 fețe.

Semi-muchiile delimitează frontiere ale fețelor, fața delimitată fiind la stânga.

Dat un poligon (eventual cu goluri):



# Exemplu



Subdiviziunea din figură are 6 vârfuri, 12 semi-muchii, 3 fețe.

Semi-muchiile delimitează frontiere ale fețelor, fața delimitată fiind la stânga.

Dat un poligon (eventual cu goluri):

- ▶ **frontieră exterioară**, care poate fi parcursă cu ajutorul semi-muchiilor astfel încât poligonul să fie la stânga frontierei, iar virajele convexe să fie la stânga,
- ▶ **frontieră interioară** (dacă există goluri), caz în care poligonul este tot la stânga, dar virajele în vârfurile convexe sunt la dreapta.

# Subdiviziuni planare

- Conceptul de subdiviziune planară: vârfuri, muchii, fețe.

## Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înălțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).

# Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlănțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).
  - ▶ **Vârf**  $v$ : coordonatele lui  $v$  în  $\text{Coordinates}(v)$ , pointer  $\text{IncidentEdge}(v)$  spre o muchie orientată care are  $v$  ca origine

# Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlănțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).
  - ▶ **Vârf**  $v$ : coordonatele lui  $v$  în  $\text{Coordinates}(v)$ , pointer  $\text{IncidentEdge}(v)$  spre o muchie orientată care are  $v$  ca origine
  - ▶ **Față**  $f$ : pointer  $\text{OuterComponent}(f)$  spre o muchie orientată corespunzătoare frontierei externe (pentru fața nemărginită este NULL); listă  $\text{InnerComponents}(f)$ , care conține, pentru fiecare gol, un pointer către una dintre muchiile orientate de pe frontieră

# Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlănțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).
  - ▶ **Vârf**  $v$ : coordonatele lui  $v$  în  $\text{Coordinates}(v)$ , pointer  $\text{IncidentEdge}(v)$  spre o muchie orientată care are  $v$  ca origine
  - ▶ **Față**  $f$ : pointer  $\text{OuterComponent}(f)$  spre o muchie orientată corespunzătoare frontierei externe (pentru fața nemărginită este NULL); listă  $\text{InnerComponents}(f)$ , care conține, pentru fiecare gol, un pointer către una dintre muchiile orientate de pe frontieră
  - ▶ **Muchie orientată**  $\vec{e}$ : pointer  $\text{Origin}(\vec{e})$ , pointer  $\text{Twin}(\vec{e})$  pointer  $\text{IncidentFace}(\vec{e})$ , pointer  $\text{Next}(\vec{e})$ , pointer  $\text{Prev}(\vec{e})$ .

# Subdiviziuni planare

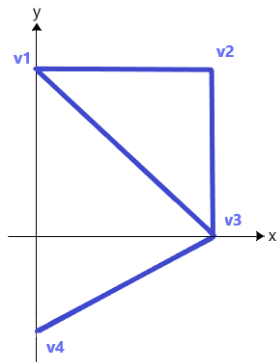
- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlănțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).
  - ▶ **Vârf**  $v$ : coordonatele lui  $v$  în  $\text{Coordinates}(v)$ , pointer  $\text{IncidentEdge}(v)$  spre o muchie orientată care are  $v$  ca origine
  - ▶ **Față**  $f$ : pointer  $\text{OuterComponent}(f)$  spre o muchie orientată corespunzătoare frontierei externe (pentru fața nemărginită este NULL); listă  $\text{InnerComponents}(f)$ , care conține, pentru fiecare gol, un pointer către una dintre muchiile orientate de pe frontieră
  - ▶ **Muchie orientată**  $\vec{e}$ : pointer  $\text{Origin}(\vec{e})$ , pointer  $\text{Twin}(\vec{e})$  pointer  $\text{IncidentFace}(\vec{e})$ , pointer  $\text{Next}(\vec{e})$ , pointer  $\text{Prev}(\vec{e})$ .
- ▶ Oricărei subdiviziuni planare  $\mathcal{S}$  i se asociază o listă de muchii dublu înlănțuite  $\mathcal{D}_{\mathcal{S}}$ .

# Subdiviziuni planare

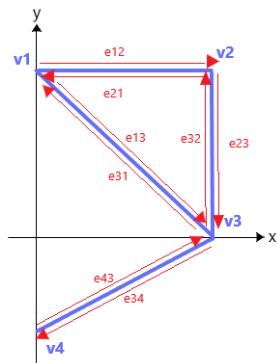
- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlănțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).
  - ▶ **Vârf**  $v$ : coordonatele lui  $v$  în  $\text{Coordinates}(v)$ , pointer  $\text{IncidentEdge}(v)$  spre o muchie orientată care are  $v$  ca origine
  - ▶ **Față**  $f$ : pointer  $\text{OuterComponent}(f)$  spre o muchie orientată corespunzătoare frontierei externe (pentru fața nemărginită este NULL); listă  $\text{InnerComponents}(f)$ , care conține, pentru fiecare gol, un pointer către una dintre muchiile orientate de pe frontieră
  - ▶ **Muchie orientată**  $\vec{e}$ : pointer  $\text{Origin}(\vec{e})$ , pointer  $\text{Twin}(\vec{e})$  pointer  $\text{IncidentFace}(\vec{e})$ , pointer  $\text{Next}(\vec{e})$ , pointer  $\text{Prev}(\vec{e})$ .
- ▶ Oricărei subdiviziuni planare  $\mathcal{S}$  i se asociază o listă de muchii dublu înlănțuite  $\mathcal{D}_{\mathcal{S}}$ .
- ▶ **Obs.** Explicați cum, folosind pointerii de mai sus: (i) poate fi parcursă frontiera exterioară / interioară a unei fețe (a unui poligon); (ii) pot fi găsite toate semi-muchiile incidente cu un vârf.



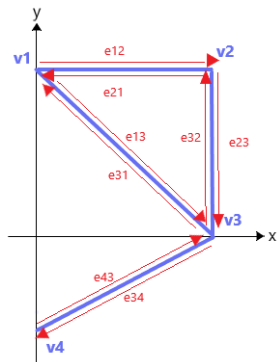
# Exemplu



## Exemplu



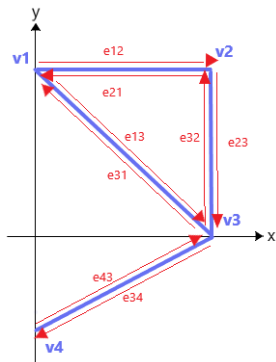
## Exemplu



| Vârf  | Coords    | Inc. | Edge     |
|-------|-----------|------|----------|
| $v_1$ | $(0, 4)$  |      | $e_{12}$ |
| $v_2$ | $(4, 4)$  |      | $e_{23}$ |
| $v_3$ | $(4, 0)$  |      | $e_{34}$ |
| $v_4$ | $(0, -2)$ |      | $e_{43}$ |

| Față           | Outer Comp. | Inner Comp(s). |
|----------------|-------------|----------------|
| (ext.) $f_1$   | NULL        | $e_{12}$       |
| $(\Delta) f_2$ | $e_{21}$    | NULL           |

## Exemplu



| Vârf  | Coords  | Inc. | Edge     |
|-------|---------|------|----------|
| $v_1$ | (0, 4)  |      | $e_{12}$ |
| $v_2$ | (4, 4)  |      | $e_{23}$ |
| $v_3$ | (4, 0)  |      | $e_{34}$ |
| $v_4$ | (0, -2) |      | $e_{43}$ |

| Față               | Outer Comp. | Inner Comp(s). |
|--------------------|-------------|----------------|
| (ext.) $f_1$       | NULL        | $e_{12}$       |
| ( $\Delta$ ) $f_2$ | $e_{21}$    | NULL           |

| Semimuchie | Origin | Twin     | Inc.  | Face | Next     | Prev     |
|------------|--------|----------|-------|------|----------|----------|
| $e_{12}$   | $v_1$  | $e_{21}$ | $f_1$ |      | $e_{23}$ | $e_{31}$ |
| $e_{23}$   | $v_2$  | $e_{32}$ | $f_1$ |      | $e_{34}$ | $e_{12}$ |
| $e_{34}$   | $v_3$  | $e_{43}$ | $f_1$ |      | $e_{43}$ | $e_{23}$ |
| ...        |        |          |       |      |          |          |

# Scurt rezumat

- Conceptul de diagramă Voronoi. Aspecte cantitative legate de numărul de vârfuri/muchii/muchii de tip semidreaptă (și la seminar).

# Scurt rezumat

- ▶ Conceptul de diagramă Voronoi. Aspecte cantitative legate de numărul de vârfuri/muchii/muchii de tip semidreaptă (și la seminar).
- ▶ Legătura cu triangulările unghiular optime.

# Scurt rezumat

- ▶ Conceptul de diagramă Voronoi. Aspecte cantitative legate de numărul de vârfuri/muchii/muchii de tip semidreaptă (și la seminar).
- ▶ Legătura cu triangulările unghiular optime.
- ▶ Algoritmul lui Fortune.

# Scurt rezumat

- ▶ Conceptul de diagramă Voronoi. Aspecte cantitative legate de numărul de vârfuri/muchii/muchii de tip semidreaptă (și la seminar).
- ▶ Legătura cu triangulările unghiular optime.
- ▶ Algoritmul lui Fortune.
- ▶ Conceptele de subdiviziune planară și de *Doubly Connected Edge List*.



## Alte referințe

*Algorithms for Competitive Programming*: Diagrame Voronoi și triangulări Delaunay

B. Kornberger, *GeomSoftware*

*CGAL*: Triangulări Delaunay și diagrame Voronoi (detalii, implementare)

ICPC World Finals 2018, Problem G Panda Preserve: (i) [enunț](#), (ii) [indicații soluție](#)